AI SEARCH ALGORITHMS

Questions and Answers: Backtracking, BFS, Heuristics, and Online Search

Question 1: Backtracking Search — Doctor Shift Assignment

Question: Assign doctors D1, D2, and D3 to Morning, Afternoon, and Night shifts. D1 cannot work Night; D2 must work before D3; each shift has one doctor; D3 cannot work Morning; and each doctor must receive exactly one shift. Use backtracking, show intermediate steps and constraint checks, and give the final schedule.

Answer

Variables: D1, D2, D3. **Domains:** {Morning (M), Afternoon (A), Night (N)}. The ordering constraint means the shift assigned to D2 must be earlier than the shift assigned to D3.

Backtracking steps and checks

1. **Step 1:** Assign D1 = Morning. Check: D1 is not assigned Night, so this is valid.

2. **Step 2:** Try D2 = Afternoon. Check: D2 and D1 have different shifts, so the one-doctor-per-shift rule is satisfied so far.

3. **Step 3:** Try D3 = Night. Check: D3 is not Morning, all three shifts are different, and D2 (Afternoon) is before D3 (Night). All constraints are satisfied.

4. **Completion check:** Every doctor has exactly one shift and every shift has exactly one doctor. No backtracking is needed on this branch.

Final valid schedule

Shift	Assigned doctor
Morning	D1
Afternoon	D2
Night	D3

Note: “D2 must work before D3” is interpreted as D2 being assigned an earlier shift than D3. The schedule above satisfies this condition.

Question 2: Grid Search — BFS, Priority Queue, and Manhattan Distance

Question: A robot must travel on a 5×5 grid from S to G. It can move Up, Down, Left, or Right; each move costs 1; obstacles cannot be crossed; and Manhattan distance is the heuristic. Show node expansion, priority queue updates, and the optimal path and cost using BFS.

Answer

Important clarification: Breadth-First Search (BFS) uses a FIFO queue and does not use a heuristic or priority queue. Manhattan distance is normally used by Greedy Best-First Search or A*. To honor the request for BFS, the worked example below uses BFS and also lists Manhattan distances for reference. Since every move costs 1, BFS returns an optimal path.

Example 5×5 grid (coordinates are row, column; rows and columns start at 1). The obstacle locations are assumed for demonstration because the question does not specify them.

S (1,1)	.	.	#	.
#	#	.	#	.
.	.	.	.	.
.	#	#	#	.
.	.	.	G (5,4)	.

Legend: S = start, G = goal, # = obstacle, . = free cell.

Manhattan distance: h(r,c) = |r – 5| + |c – 4|, because the goal is at (5,4).

BFS expansion order and queue updates

Neighbour order used for reproducibility: Right, Down, Left, Up. The queue shown is the FIFO queue after each node is expanded. Already visited cells and obstacles are skipped.

Expanded node	New nodes added	Queue after expansion
(1,1)	(1,2)	(1,2)
(1,2)	(1,3)	(1,3)
(1,3)	(2,3)	(2,3)
(2,3)	(3,3)	(3,3)
(3,3)	(3,4), (3,2)	(3,4), (3,2)
(3,4)	(3,5)	(3,2), (3,5)
(3,2)	(3,1)	(3,5), (3,1)
(3,5)	(4,5), (2,5)	(3,1), (4,5), (2,5)
(3,1)	(4,1)	(4,5), (2,5), (4,1)
(4,5)	(5,5)	(2,5), (4,1), (5,5)
(2,5)	(1,5)	(4,1), (5,5), (1,5)
(4,1)	(5,1)	(5,5), (1,5), (5,1)
(5,5)	(5,4)	(1,5), (5,1), (5,4)
(1,5)	None	(5,1), (5,4)
(5,1)	(5,2)	(5,4), (5,2)
(5,4)	None	(5,2)

Optimal path found: (1,1) → (1,2) → (1,3) → (2,3) → (3,3) → (3,4) → (3,5) → (4,5) → (5,5) → (5,4)

Path cost: 9 moves × 1 = 9.

Why optimal? BFS explores nodes in increasing number of moves. With equal move costs, the first time it reaches G, the path has the minimum number of moves.

Priority queue note: There is no priority queue in standard BFS. If the assignment specifically requires priority queue updates using Manhattan distance, use A* with f(n) = g(n) + h(n) instead. A* is a different algorithm from BFS.

Question 3: Autonomous Rescue Robot — Online Search in a Partially Observable Grid

Question: A rescue robot must reach a survivor from S to G in a building grid. It can move up, down, left, or right; obstacles block cells; it can sense only adjacent cells; each move costs 1; risky zones add cost 2; the robot must minimize cost and update knowledge dynamically. Analyze the constraints, propose a search strategy, explain AI concepts, and justify the approach.

Answer

a) Analysis of constraints

• **Four-direction movement:** The robot can move only to adjacent cells, so diagonal shortcuts are not allowed.

• **Obstacles:** Blocked cells must be excluded from the available actions.

• **Partial observability:** The robot cannot see the entire building at once. It maintains an internal map of known free, blocked, risky, and unknown cells.

• **Limited sensing:** Before moving, it checks adjacent cells and updates its map with new observations.

• **Unequal movement costs:** A normal move costs 1. Entering a risky cell costs 1 + 2 = 3, so the shortest route by number of steps may not be the cheapest route.

• **Online updates:** If a newly observed obstacle blocks the planned route, the robot must replan using its latest map.

b) Suitable solution: Online sensing with Uniform Cost Search (UCS)

UCS is suitable for finding a minimum-cost route when action costs are non-negative and may differ. Because the environment is only partly known, the robot uses UCS on its current map, senses as it moves, and replans whenever new information changes the route.

1. **Initialize:** Put S into a priority queue with path cost g(S) = 0. Store the known map and mark unobserved cells as unknown.

2. **Sense adjacent cells:** Identify which neighboring cells are free, blocked, or risky. Update the map; do not enter a known obstacle.

3. **Expand the lowest-cost node:** Remove the frontier cell with the smallest accumulated cost g(n).

4. **Calculate transition cost:** Entering a normal cell costs 1; entering a risky cell costs 3 (movement 1 + risk penalty 2). Update a neighbor when a lower-cost route is found.

5. **Choose and move:** Follow the first step on the lowest-cost known route. Since the map is incomplete, do not assume unseen cells are safe.

6. **Repeat sensing and planning:** After movement, sense adjacent cells and re-run or update the search if new obstacles or risks are discovered.

7. **Stop:** When the robot reaches G, report the route and total cost. If G is not reachable on the known map, continue exploring safe reachable frontier cells.

Cost formula: Total cost = number of moves + 2 × (number of risky cells entered). For example, a route with 6 moves and 1 risky cell costs 6 + 2 = 8.

Safety rule: Never deliberately move into a known blocked cell. For unknown cells, use cautious exploration and replan immediately when an obstacle or hazard is detected.

c) AI concepts demonstrated

• **Intelligent agent:** The robot senses its surroundings, maintains a map, selects an action, moves, and repeats the cycle.

• **Rationality:** A rational action uses available information to minimize expected/known path cost while respecting safety constraints. It avoids known obstacles and prefers lower-cost safe routes.

• **Environment type:** Partially observable, sequential, dynamic in the robot's knowledge, spatial/grid-based, and stochastic or uncertain if newly discovered conditions are not known in advance. Actions have deterministic costs as specified, unless the scenario adds uncertainty.

• **Performance measure:** Reach the survivor safely while minimizing total movement and risk cost.

d) Why this approach is suitable

1. UCS handles different non-negative costs correctly; ordinary BFS is not sufficient when risky cells cost more.

2. The priority queue ensures that the least accumulated-cost frontier node is expanded first.

3. Online sensing allows the robot to work without a complete map at the beginning.

4. Repeated map updates and replanning let it respond to newly discovered obstacles and risky zones.

5. The approach is easy to explain and verify because every move has an explicit cost. In a large environment, an incremental planner such as D* Lite could reduce repeated planning work.

Conclusion: Use an online search agent that senses nearby cells and applies Uniform Cost Search to the current map. This balances limited information, obstacle avoidance, and minimum-cost navigation.